

Além da BST

Quando dois filhos não são suficientes.

1

Limitações
da BST

2

Árvores
N-árias

3

Exercício
N-árias

4

Tries

5

Exercício
Tries

sistema de arquivos

Toda pasta no seu computador pode conter qualquer número de subpastas.

Como você usaria uma BST para representar essa estrutura?

Pense: cada nó da BST tem exatamente dois filhos. Como representar uma pasta com 10 subpastas? E com 0?

O problema:

A BST assume exatamente dois filhos por nó – esquerdo e direito. Uma estrutura hierárquica com número variável de filhos não se encaixa nesse modelo.

Menu de categorias

Eletrônicos tem: Celulares, Computadores, TVs, Áudio, Câmeras.

Celulares tem: Android, iPhone, Acessórios.

Cada categoria tem subcategorias próprias, e o número varia.

Como você usaria uma BST para representar esse menu de categorias?

Pense: a BST organiza por comparação numérica. Como comparar categorias? Como um nó teria 5 filhos?

O problema:

A BST foi projetada para comparar valores e organizar dois filhos. Hierarquias reais têm número arbitrário de filhos e precisam de uma estrutura diferente.

Autocomplete do Google

Ao digitar "prog" o Google sugere instantaneamente: programação, programador programar. Isso acontece a cada letra digitada, em milissegundos.

Como você usaria uma BST para buscar todas as palavras que começam com "prog"?

O problema:

A BST encontra um valor em $O(\log n)$. Encontrar todos os valores com um prefixo comum exige percorrer a árvore inteira – $O(n)$. Para autocomplete em tempo real, isso é inviável.

Situações 1 e 2 → precisamos de uma árvore com N filhos por nó.

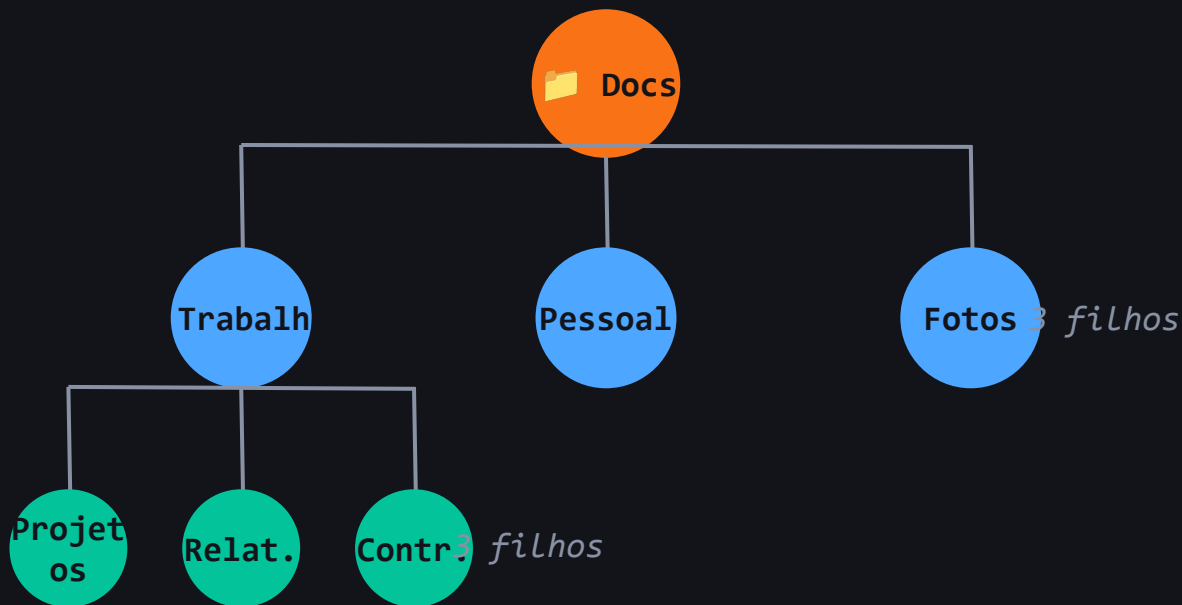
Situação 3 → precisamos de uma árvore que navega por caracteres.

Árvore N-ária – cada nó pode ter N filhos

Definição:

Uma árvore N-ária é uma generalização da árvore binária. Em vez de no máximo dois filhos por nó, cada nó pode ter quantos filhos forem necessários.

Exemplo – sistema de arquivos:



Como implementar a estrutura do nó

A diferença fundamental em relação à BST: em vez de dois ponteiros fixos (esq e dir), o nó tem uma lista de ponteiros filhos – de tamanho variável.

BST:

```
struct No {  
    int valor;  
    No* esq;  
    No* dir;  
};
```

N-ária:

```
struct No {  
    string valor;  
    vector<No*> filhos;  
};
```

A chave:

`vector<No*> filhos` – um vetor de ponteiros. Adicionar um filho é fazer `push_back`. Percorrer os filhos é um `for` loop. Não há limite fixo de filhos por nó.

`#include<vector>` e `#include<string>`

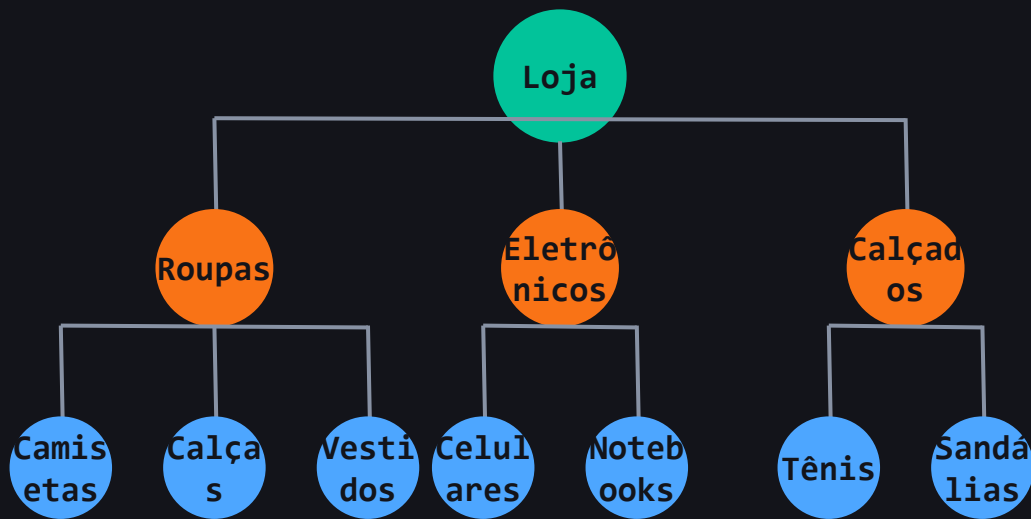
Criar nó e adicionar filhos

```
struct No {  
    string valor;  
    vector<No*> filhos;  
    No(string v) { valor = v; }  
};  
  
// Adicionar filho a um nó  
void adicionarFilho(No* pai, No* filho) {  
    pai->filhos.push_back(filho);  
}  
  
// Percorrer a árvore (pré-ordem)  
void percorrer(No* raiz, int nivel) {  
    if (raiz == nullptr) return;  
    for (int i = 0; i < nivel; i++) cout << "  ";  
    cout << raiz->valor << endl;  
    for (No* filho : raiz->filhos)  
        percorrer(filho, nivel + 1);  
}
```

O percurso usa recursão – para cada nó, percorre todos os filhos. O parâmetro nível controla a indentação para visualizar a hierarquia.

Exercício – menu de categorias

Uma loja online organiza seus produtos em categorias e subcategorias. Implemente a estrutura abaixo usando uma árvore N-ária:



➡ Implemente a estrutura acima e crie uma função que exiba o menu com indentação.

Saída esperada: Loja → Roupas → Camisetas → Calças → Vestidos →
Eletrônicos → ...

Menu de categorias

```
int main() {  
    No* loja = new No("Loja");  
  
    No* roupas = new No("Roupas");  
    adicionarFilho(roupas, new No("Camisetas"));  
    adicionarFilho(roupas, new No("Calcas"));  
    adicionarFilho(roupas, new No("Vestidos"));  
  
    No* eletronicos = new No("Eletronicos");  
    adicionarFilho(eletronicos, new No("Celulares"));  
    adicionarFilho(eletronicos, new No("Notebooks"));  
  
    adicionarFilho(loja, roupas);  
    adicionarFilho(loja, eletronicos);  
    adicionarFilho(loja, new No("Calcados"));  
    percorrer(loja, 0);  
    return 0;  
}
```

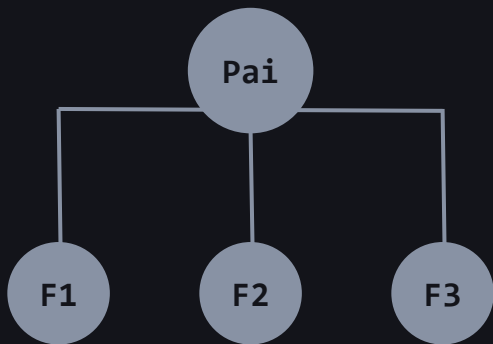
Uma segunda forma de implementar a N-ária

Na implementação com vetor, o pai armazena ponteiros para todos os filhos. O vetor cresce dinamicamente – custo de memória proporcional ao número de filhos.

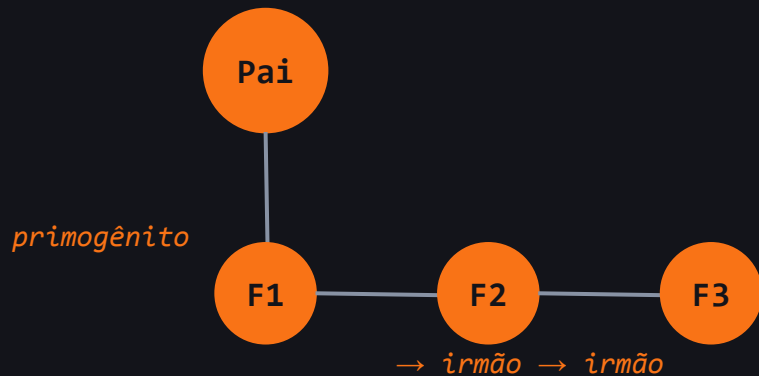
Alternativa:

O pai aponta apenas para o primogênito. Cada filho aponta para o próximo irmão. Tamanho do nó sempre fixo – dois ponteiros.

Vetor de filhos:



Primogênito + Irmão:



Estrutura do nó – dois ponteiros fixos

Em vez de um vetor dinâmico, cada nó tem exatamente dois ponteiros – um para o filho mais velho e outro para o próximo irmão. O tamanho do nó é sempre o mesmo, independente de quantos filhos ele tiver.

Comparação:

Antes – vetor:

```
struct No {  
    string valor;  
    vector<No*> filhos;  
};
```

Agora – dois ponteiros:

```
struct No {  
    string valor;  
    No* primogenito;  
    No* proximoIrmao;  
    No(string v) {  
        valor = v;  
        primogenito = nullptr;  
        proximoIrmao = nullptr;  
    }  
};
```

Adicionar filho:

```
void adicionarFilho(No* pai, No* filho) {
    if (pai->primogenito == nullptr) {
        pai->primogenito = filho; // primeiro filho
        return;
    }
    // Percorre a lista de irmãos até o último
    No* atual = pai->primogenito;
    while (atual->proximoIrmao != nullptr)
        atual = atual->proximoIrmao;
    atual->proximoIrmao = filho; // adiciona ao final
}
```

Percorrer:

```
void percorrer(No* raiz, int nivel) {
    if (raiz == nullptr) return;
    for (int i = 0; i < nivel; i++) cout << "  ";
    cout << raiz->valor << endl;
    percorrer(raiz->primogenito, nivel + 1); // desce filhos
    percorrer(raiz->proximoIrmao, nivel);    // percorre irmãos
}
```

Por que o percurso funciona assim?

A lógica recursiva tem dois movimentos distintos – e os níveis são diferentes em cada um:

```
percorrer(primogenito, nivel + 1)
```

Desce um nível na hierarquia – entra nos filhos. O nível aumenta porque os filhos estão mais profundos na árvore.

```
percorrer(proximoIrmao, nivel)
```

Permanece no mesmo nível – percorre os irmãos. O nível não muda porque irmãos estão na mesma profundidade.

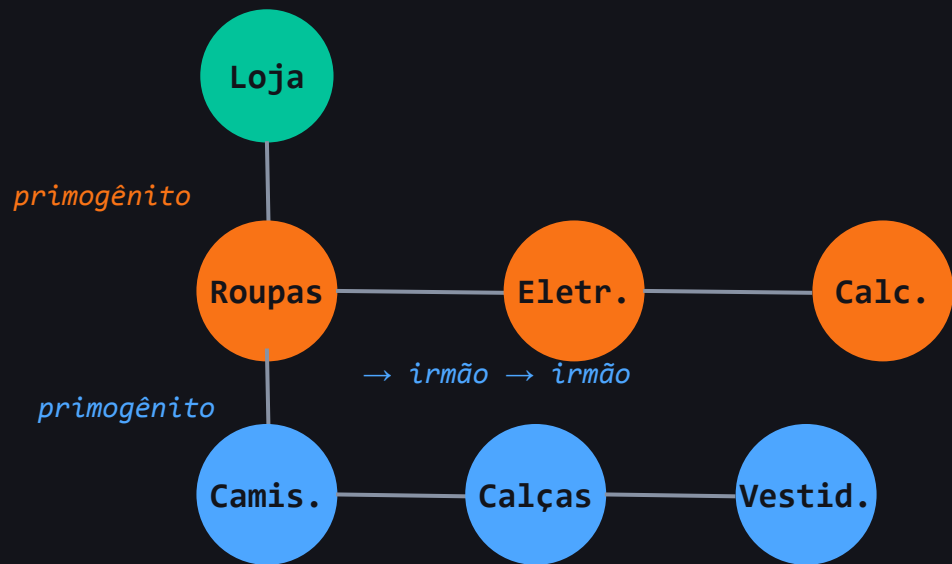
Rastreando com Loja → Roupas, Eletrônicos, Calçados:

```
percorrer(Loja, 0)      → imprime 'Loja'
  percorrer(Roupas, 1)  → imprime '  Roupas'
    percorrer(nullptr, 2) → retorna
      percorrer(Eletrônicos, 1) → imprime '    Eletrônicos'
        percorrer(Calçados, 1) → imprime '      Calçados'
```

Exercício – reconstrua o menu de categorias

Implemente a mesma estrutura do exercício anterior usando a abordagem primogênito + irmão – sem usar vector.

Estrutura esperada:



✎ Implemente a estrutura acima usando No* primogenito e No* proximoIrmão. Use a função percorrer para exibir o menu com indentação.

Resposta – menu de categorias

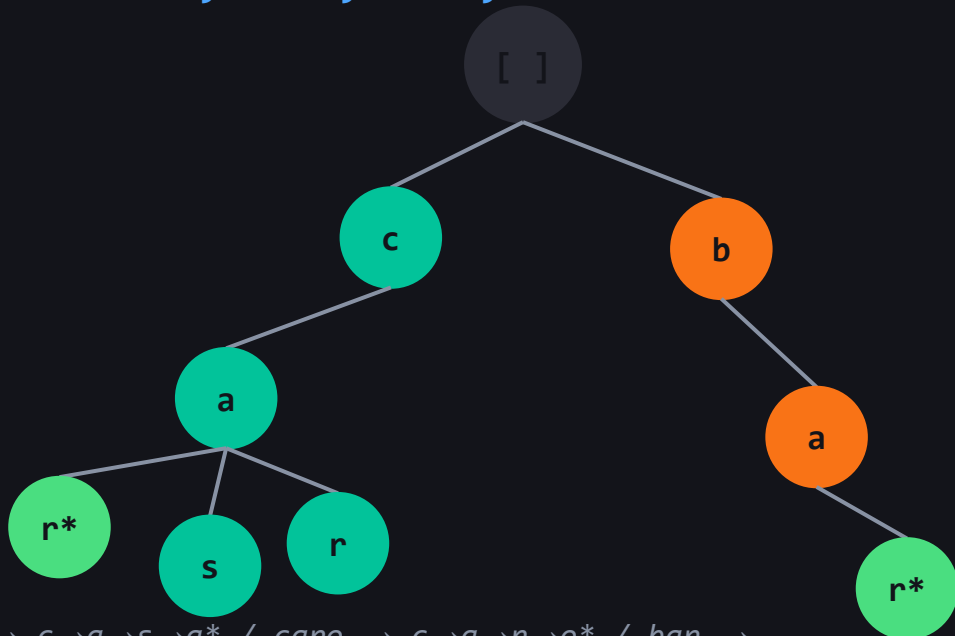
```
int main() {  
    No* loja          = new No("Loja");  
    No* roupas        = new No("Roupas");  
    No* eletronicos = new No("Eletronicos");  
    No* calcados      = new No("Calcados");  
    No* camisetas     = new No("Camisetas");  
    No* calcas        = new No("Calcas");  
    No* vestidos      = new No("Vestidos");  
  
    // Conectar irmãos de Loja  
    loja->primogenito    = roupas;  
    roupas->proximoIrmao = eletronicos;  
    eletronicos->proximoIrmao = calcados;  
  
    // Conectar filhos de Roupas  
    roupas->primogenito    = camisetas;  
    camisetas->proximoIrmao = calcas;  
    calcas->proximoIrmao    = vestidos;  
  
    percorrer(loja, 0);  
    return 0;  
}
```

Trie – a árvore que navega por letras

Definição:

Uma Trie é uma árvore N-ária onde cada nó representa um caractere. O caminho da raiz até um nó forma um prefixo ou uma palavra completa.

Exemplo – palavras: car, casa, caro, bar



$car \rightarrow c \rightarrow a \rightarrow r^*$ / $casa \rightarrow c \rightarrow a \rightarrow s \rightarrow a^*$ / $caro \rightarrow c \rightarrow a \rightarrow r \rightarrow o^*$ / $bar \rightarrow b \rightarrow a \rightarrow r^*$

**fim de palavra*

Por que a Trie resolve o autocomplete?

Você digita "ca" – e quer todas as palavras que começam com "ca".

Com BST

Você precisa percorrer todos os nós da árvore e verificar se cada palavra começa com "ca". Complexidade: $O(n \times m)$ – n palavras, m tamanho médio.

Com Trie

Você desce pelo nó 'c', depois pelo nó 'a' – dois passos. A partir daí, todos os descendentes são palavras com prefixo "ca".
Complexidade: $O(m)$ para chegar ao prefixo + $O(k)$ para listar os k resultados.

A Trie não percorre – ela navega. Cada letra digitada é um passo na árvore, não uma busca.

Estrutura do nó de uma Trie

Cada nó representa um caractere. Para o alfabeto português (26 letras), cada nó tem até 26 filhos – um por letra possível. A maioria será nullptr.

```
#define TAMANHO 26

struct NoTrie {
    NoTrie* filhos[TAMANHO]; // um por letra
    bool fimDePalavra;       // marca fim de palavra

    NoTrie() {
        fimDePalavra = false;
        for (int i = 0; i < TAMANHO; i++)
            filhos[i] = nullptr;
    }
};
```

O índice do filho é calculado pelo caractere: 'a'=0, 'b'=1, 'c'=2...
usando `c - 'a'`.

Inserção e busca na Trie

```
// Inserir uma palavra
void inserir(NoTrie* raiz, string palavra) {
    NoTrie* atual = raiz;
    for (char c : palavra) {
        int idx = c - 'a';
        if (!atual->filhos[idx])
            atual->filhos[idx] = new NoTrie();
        atual = atual->filhos[idx];
    }
    atual->fimDePalavra = true;
}
```

```
// Buscar uma palavra
bool buscar(NoTrie* raiz, string palavra) {
    NoTrie* atual = raiz;
    for (char c : palavra) {
        int idx = c - 'a';
        if (!atual->filhos[idx]) return false;
        atual = atual->filhos[idx];
    }
    return atual->fimDePalavra;
}
```

Exercício – sistema de autocomplete

Um sistema de busca precisa sugerir palavras enquanto o usuário digita. Implemente um autocomplete simples usando uma Trie.

Cadastrar palavras

- 1 Inserir na Trie as palavras: casa, carro, carta, capa, bar, barra, base.

Verificar existência

- 2 Buscar se "carro" existe. Buscar se "capo" existe. Exibir o resultado de cada busca.

Listar por prefixo

- 3 Implementar uma função que recebe um prefixo e imprime todas as palavras cadastradas que começam com ele.

Dica para a tarefa 3: percorra recursivamente a partir do nó onde o prefixo termina.

```

// Coleta palavras a partir de um nó
void coletar(NoTrie* no, string prefixo,
             vector<string>& resultado) {
    if (no->fimDePalavra)
        resultado.push_back(prefixo);
    for (int i = 0; i < TAMANHO; i++) {
        if (no->filhos[i])
            coletar(no->filhos[i],
                    prefixo + char('a'+i), resultado);
    }
}

// Buscar por prefixo
void autocomplete(NoTrie* raiz, string prefixo) {
    NoTrie* atual = raiz;
    for (char c : prefixo) {
        int idx = c - 'a';
        if (!atual->filhos[idx]) return; // prefixo não existe
        atual = atual->filhos[idx];
    }
    vector<string> res;
    coletar(atual, prefixo, res);
    for (string p : res) cout << p << endl;
}

```